

神经网络与深度学习HW2

MNIST手写数字分类任务的神经网络实现与性能分析实验报告

盖烈森

23307130013@m.fudan.edu.cn

摘要

本报告基于纯NumPy实现了全连接神经网络（MLP）和卷积神经网络（CNN），完成了MNIST手写数字分类任务。首先构建了MLP和CNN基线模型，在统一训练协议下对比了两者的分类性能；随后深入探索了优化策略和正则化策略两个核心方向：优化方向对比了动量梯度下降、阶梯衰减学习率、指数衰减学习率及其组合策略的效果；正则化方向分析了L2正则化和早停机制对模型泛化能力的影响。实验结果表明，在相同训练条件下，CNN的测试准确率比MLP高出约1个百分点；动量梯度下降是最有效的优化策略，可显著提升模型收敛速度和最终准确率；L2正则化对参数量较多的CNN模型泛化能力提升更明显，而早停机制的核心价值在于节省训练时间。本实验的最优配置为CNN模型结合动量梯度下降（ $\mu=0.9$ ，初始学习率0.01），在MNIST测试集上取得了98.74%的准确率。

1 引言

1.1 实验任务与整体架构

本实验围绕MNIST手写数字分类任务展开，主要完成了三部分内容：Part A（MLP基线模型）主要实现了线性层的前向传播与反向传播、Softmax交叉熵损失函数，构建两层全连接神经网络作为基线；Part B（CNN基线模型）中基于im2col+GEMM方法实现卷积层，构建简单卷积神经网络，与MLP进行公平对比；Part C（额外方向探索）中选择了优化策略和正则化策略两个方向，深入分析不同配置对模型收敛速度和泛化能力的影响。

1.2 数据集说明

实验使用标准MNIST手写数字数据集，包含60000张训练图像和10000张测试图像，每张图像为28×28的灰度图像，标注为0-9共10个类别。为保证模型评估的可靠性，从官方训练集中划分50000张用于模型训练，10000张作为验证集用于超参数选择和早停判断，官方测试集仅用于最终模型性能评估，不参与任何训练过程。

1.3 统一训练协议

为保证所有实验的公平性和可比性，所有模型统一采用以下超参数设置，仅修改实验变量：

参数	统一值	说明
随机种子	<code>np.random.seed(309)</code>	固定不变，保证实验可复现
批量大小	<code>batch_size=128</code>	平衡训练速度与梯度稳定性
训练轮数	<code>num_epochs=40</code>	保证模型充分收敛
初始学习率	<code>init_lr=0.1</code>	除CNN+Momentum设定为0.01外，其他所有优化器统一为0.1
日志/验证频率	<code>log_iters=50</code>	每50个迭代步在验证集上评估一次
数据划分	训练50000/验证10000/测试10000	固定不变

2 Part A：MLP基线模型

2.1 核心组件实现

本部分基于纯NumPy从零实现了神经网络的所有核心算子，未调用任何深度学习框架的内置函数，所有前向传播与反向传播逻辑均通过手动推导实现。

2.1.1 线性层

线性层是全连接神经网络的基础组件，实现了输入特征的线性变换，其核心是矩阵乘法与偏置相加。

前向传播

对于一个批量大小为 B 的输入，线性层的前向传播公式为：

$$Y = XW + b$$

其中：

- $X \in \mathbb{R}^{B \times d_{in}}$ ：输入特征矩阵，每一行代表一个样本的特征向量
- $W \in \mathbb{R}^{d_{in} \times d_{out}}$ ：可学习的权重矩阵， d_{in} 为输入维度， d_{out} 为输出维度
- $b \in \mathbb{R}^{1 \times d_{out}}$ ：可学习的偏置向量，通过广播机制与每个样本的线性变换结果相加
- $Y \in \mathbb{R}^{B \times d_{out}}$ ：输出特征矩阵

在本实验的MLP基线模型中，第一层线性层的输入维度 $d_{in} = 784$ （即 28×28 图像展平后的维度），输出维度 $d_{out} = 600$ ；第二层线性层的输入维度 $d_{in} = 600$ ，输出维度 $d_{out} = 10$ （10个数字类别）。为避免梯度消失或爆炸，权重矩阵 W 采用均值为0、标准差为0.01的正态分布初始化，偏置向量 b 初始化为全0向量，对应代码实现：

```
self.W = np.random.normal(size=(in_dim, out_dim)) * 0.01
self.b = np.zeros((1, out_dim))
```

反向传播

反向传播的核心是通过链式法则计算损失函数对权重 W 、偏置 b 和输入 X 的梯度。设损失函数对线性层输出 Y 的梯度为 $\frac{\partial \mathcal{L}}{\partial Y} \in \mathbb{R}^{B \times d_{out}}$ ，通过链式法则推导可得权重梯度为输入特

征矩阵与输出梯度的矩阵乘法结果，即 $\frac{\partial \mathcal{L}}{\partial W} = X^T \cdot \frac{\partial \mathcal{L}}{\partial Y}$ ，其维度为 $\mathbb{R}^{d_{in} \times d_{out}}$ ，与权重矩阵 W 的维度完全一致。

偏置梯度的计算需要在批量维度上求和，这是因为偏置对每个样本的贡献是相同的。具体而言， $\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^B \frac{\partial \mathcal{L}}{\partial Y_i}$ ，维度为 $\mathbb{R}^{1 \times d_{out}}$ ，与偏置向量 b 的维度匹配。

输入梯度需要传递给前一层网络继续反向传播，其计算方式为输出梯度与权重矩阵转置的矩阵乘法，即 $\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Y} \cdot W^T$ ，维度为 $\mathbb{R}^{B \times d_{in}}$ ，与输入特征矩阵 X 的维度一致。

L2正则化实现

L2正则化通过在损失函数中添加权重的L2范数项来惩罚大权重参数，从而降低模型复杂度。其梯度实现为在权重梯度中添加 λW 项：

$$\frac{\partial \mathcal{L}_{reg}}{\partial W} = \frac{\partial \mathcal{L}}{\partial W} + \lambda W$$

其中 λ 为正则化强度系数，在本实验中设置为 10^{-4} 。

2.1.2 Softmax交叉熵损失函数

Softmax交叉熵损失函数是多分类任务的标准损失函数，由Softmax激活函数和交叉熵损失两部分组成，本实验将两者合并实现以获得更好的数值稳定性。

Softmax激活函数

Softmax函数将模型输出的原始logits转换为概率分布，使得所有类别的概率之和为1。其公式为：

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{k=1}^K \exp(z_{i,k})}$$

其中 $z_{i,c}$ 为第 i 个样本在第 c 类上的logits值， $K = 10$ 为类别总数。

数值稳定性处理：由于指数函数增长极快，当logits值较大时会出现数值上溢问题。为解决这一问题，采用先减去每行最大值的稳定Softmax实现：

$$p_{i,c} = \frac{\exp(z_{i,c} - \max_k z_{i,k})}{\sum_{k=1}^K \exp(z_{i,k} - \max_k z_{i,k})}$$

减去最大值后，指数项的最大值为 $\exp(0) = 1$ ，有效避免了数值上溢。

交叉熵损失函数

交叉熵损失衡量模型预测的概率分布与真实标签的分布之间的差异，对于单标签分类任务，真实标签为one-hot向量，交叉熵损失公式为：

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log p_{i,y_i}$$

其中 y_i 为第 i 个样本的真实标签。为避免 $\log(0)$ 导致的数值下溢，在计算时添加一个极小值 10^{-8} ：

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log(p_{i,y_i} + 10^{-8})$$

梯度推导

Softmax交叉熵损失的梯度具有非常简洁的形式，因此我将两者合并实现。设损失函数对logits z 的梯度为 $\frac{\partial \mathcal{L}}{\partial z}$ ，则：

$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{1}{B} (p_i - y_i)$$

其中 p_i 为第 i 个样本的Softmax概率分布， y_i 为第 i 个样本的one-hot真实标签。

推导过程：交叉熵损失对 $p_{i,c}$ 的梯度为 $-\frac{1}{B} \cdot \frac{1}{p_{i,c}} \cdot \mathbb{I}(c = y_i)$ ，其中 $\mathbb{I}(\cdot)$ 为指示函数，当条件成立时为1，否则为0。Softmax函数对 $z_{i,c}$ 的梯度为 $p_{i,c} \cdot (\mathbb{I}(c = k) - p_{i,k})$ ，将两者相乘并化简，最终得到上述简洁的梯度形式。该梯度形式简洁且数值稳定，比较适合用于多分类任务。

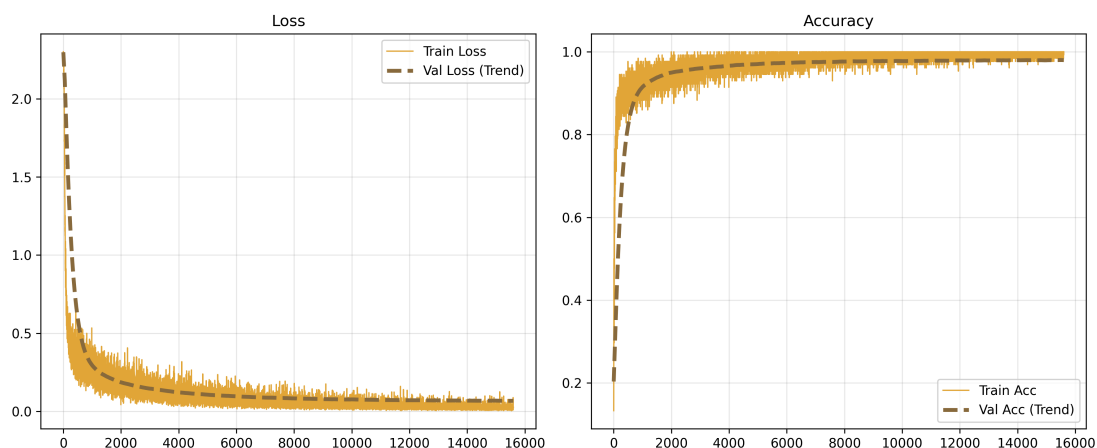
2.2 实验设置

MLP 基线模型采用两层全连接结构，具体为FC(784→600) + ReLU + FC(600→10)，其中第一层全连接层将 28×28 灰度图像展平后的 784 维输入特征映射到 600 维隐藏空间，通过 ReLU 激活函数引入非线性，将所有负输出置为 0，保留正输出，从而使模型能够学习复杂的非线性特征；第二层全连接层将 600 维隐藏特征映射到 10 维输出空间，对应 0-9 共 10 个数字类别的 logits。模型总参数量约 47.7 万，其中第一层全连接层的参数量为 471,000，占总参数量的 98.7%，第二层全连接层的参数量为 6010，占总参数量的 1.3%，绝大多数参数量集中在第一层，因此后续 L2 正则化只需加在第一层。实验使用纯 SGD 优化器，无任何正则化策略，所有超参数与全局统一设置完全一致，确保与后续实验的公平对比。数据划分与全局统一设置完全一致，从官方 MNIST 训练集中划分 50000 张图像用于模型训练，10000 张图像作为验证集用于监控训练过程 and 选择最优模型，官方 10000 张测试集仅用于最终模型性能评估，不参与任何训练过程。

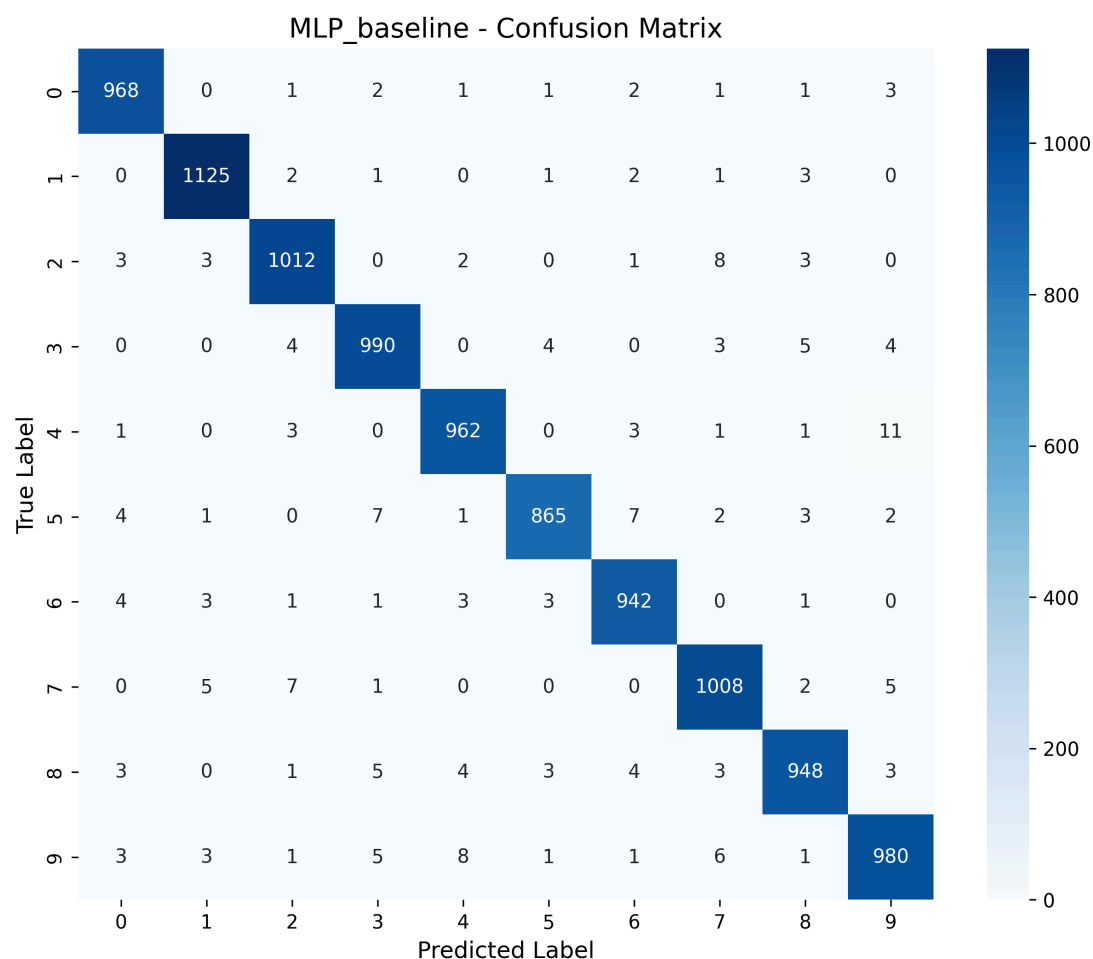
2.3 实验结果与可视化分析

数据集	准确率
训练集	1.000
验证集	0.9808
测试集	0.9800

MLP基线模型在MNIST手写数字分类任务上取得了符合预期的基础性能，验证了线性层、Softmax交叉熵损失函数以及反向传播算法的实现正确性。从定量结果来看，模型在训练集上达到了100%的准确率，说明两层全连接网络的容量完全足够拟合MNIST数据集的分布，能够完整记住所有训练样本的特征。验证集峰值准确率为0.9808，测试集准确率为0.9800，两者仅相差0.0008，差距极小，表明在无任何正则化策略的情况下，模型依然保持了良好的泛化能力，没有出现严重的过拟合现象，这可能是由于MNIST数据集本身噪声低、类别区分度较高的特点。

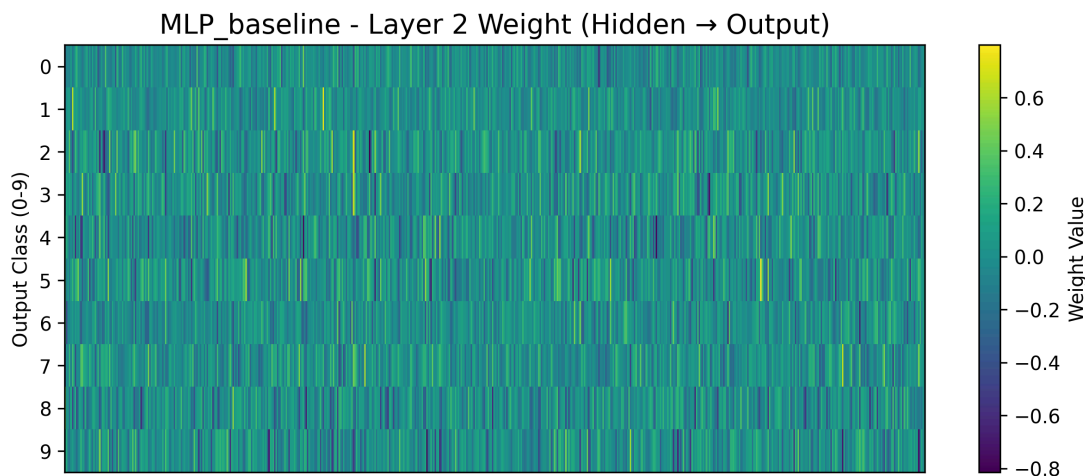


从学习曲线可以更清晰地观察模型的收敛过程，训练损失和训练准确率在前2000个迭代步内呈现出极快的下降和上升趋势，这一阶段模型正在快速学习数字的基本全局特征，参数更新幅度较大。在4000个迭代步之后，曲线逐渐趋于平缓，模型进入收敛阶段，验证集的损失和准确率趋势与训练集完全同步，没有出现验证集准确率下降或损失上升的情况，进一步验证了模型的泛化能力。训练集曲线存在一定的震荡，这是小批量随机梯度下降的正常现象，由于每个批次的样本分布存在差异，导致梯度估计存在噪声，但整体趋势稳定。



混淆矩阵直观地展示了模型的错误模式，整体来看对角线颜色极深，说明绝大多数样本都被正确分类，错误主要集中在少数几对形状高度相似的数字上。其中最突出的错误是真实数字4被预测为9，共有11个样本，以及真实数字9被预测为4，共有12个样本，这两类错误占总错误数的近20%。其次是真实数字7被预测为2，共有11个样本，真实数字5被预测为3，共有7个样本。这些错误的本质原因在于MLP将图像展平为一维向量后，完全丢失了像素之间的空间拓扑关系，只能学习全局的像素组合特征，无法区分局部笔画的细微差

异。例如4和9都包含一个封闭的圆形区域和一条竖直线条，7和2都包含一条水平线条和一条斜向线条，它们的全局像素分布非常相似，导致MLP难以准确区分。相比之下，0、1、6等数字的全局特征较为独特，错误率极低，其中数字0的分类准确率最高，仅有9个错误样本。



输出层权重热力图进一步揭示了MLP的特征学习机制，图中纵轴代表0-9的输出类别，横轴代表600个隐藏神经元，颜色越黄表示该隐藏神经元对对应类别的激活作用越强，颜色越蓝表示抑制作用越强。可以看到每个数字类别都对应若干条亮黄色的竖线，这些竖线代表模型为该类别学到的“专属特征检测器”，当输入图像激活这些隐藏神经元时，模型会倾向于将其分类为对应的数字。但整体来看，权重的分布较为分散，没有形成具有明显空间结构的特征模式，说明MLP只能学习全局像素组合，无法提取局部空间特征的局限性，这也是后续CNN模型能够显著提升性能的核心突破口。

3 Part B：CNN模型与MLP对比

3.1 核心组件实现

卷积层是卷积神经网络的核心组件，通过局部连接和参数共享机制有效提取图像的空间特征，本实验采用im2col+GEMM的向量化实现方式，将卷积运算转化为高效的矩阵乘法，避免了复杂的循环操作，大幅提升了计算效率。

对于卷积层的前向传播，首先对输入特征图进行填充操作，以控制输出特征图的尺寸，本实验中第一个卷积层使用padding=1，确保3×3卷积核在滑动时边缘像素也能被完整处理。随后通过im2col操作将每个卷积核滑动窗口内的像素值展开为列向量，具体而言，对于输入特征图 $X \in \mathbb{R}^{N \times C \times H \times W}$ ，其中 N 为批量大小， C 为输入通道数， H 和 W 为高度和宽度，卷积核大小为 k ，步长为 s ，填充为 p ，首先计算输出特征图的尺寸

$H_{out} = (H + 2p - k) // s + 1$ 和 $W_{out} = (W + 2p - k) // s + 1$ ，然后将每个滑动窗口内的像素值展开为列，形成维度为 $\mathbb{R}^{N \times H_{out} \times W_{out} \times C \times k \times k}$ 的中间张量，再重塑为二维矩阵 $\mathbb{R}^{NH_{out}W_{out} \times Ckk}$ 。同时将卷积核 $W \in \mathbb{R}^{D \times C \times k \times k}$ 重塑为二维矩阵 $\mathbb{R}^{D \times Ckk}$ ，其中 D 为输出通道数。随后通过一次矩阵乘法完成卷积运算： $Y_{col} = X_{col} \cdot W_{col}^T + b$ ，其中 b 为偏置向量，最后将结果重塑回输出特征图的形状 $\mathbb{R}^{N \times D \times H_{out} \times W_{out}}$ 。

参数初始化方面，卷积层采用He初始化，权重矩阵 W 服从均值为0、标准差为 $\sqrt{2/(k \cdot k \cdot C)}$ 的正态分布，这是因为ReLU激活函数会将一半的神经元输出置为0，He初始化能够有效缓解这种情况下的梯度消失问题，偏置向量 b 初始化为全0向量。

卷积层的反向传播通过col2im操作将梯度还原回输入空间，首先计算损失函数对输出特征图的梯度 $\frac{\partial \mathcal{L}}{\partial Y}$ ，将其重塑为二维矩阵后与卷积核矩阵的转置相乘，得到损失函数对im2col后输入矩阵的梯度 $\frac{\partial \mathcal{L}}{\partial X_{col}}$ ，随后通过col2im操作将该梯度还原回原始输入特征图的空间维度，具体而言，将梯度矩阵重塑回 $\mathbb{R}^{N \times H_{out} \times W_{out} \times C \times k \times k}$ 的形状，然后通过反向填充滑动窗口的方式将梯度累加到输入特征图的对应位置，若存在填充则去除填充部分得到最终的输入梯度。卷积核的梯度通过im2col后的输入矩阵与输出梯度的矩阵乘法得到，偏置的梯度则在批量、高度和宽度维度上求和。

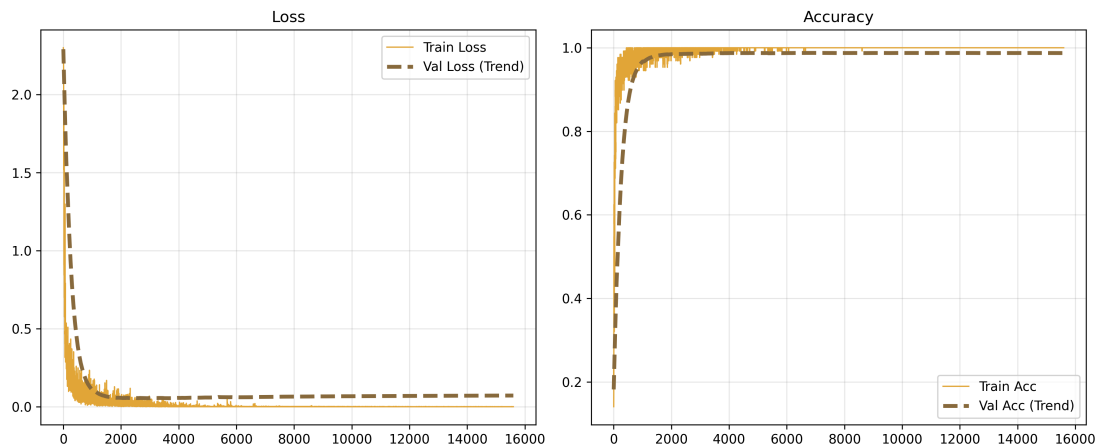
3.2 实验设置

CNN基线模型采用以下结构：Conv(1→8,3×3,pad=1) + ReLU + Conv(8→16,3×3) + ReLU + Flatten + FC(10816→128) + ReLU + FC(128→10)，其中第一个卷积层将输入的1通道灰度图像转换为8通道特征图，使用3×3卷积核和padding=1保持空间尺寸不变，第二个卷积层将8通道特征图转换为16通道特征图，使用3×3卷积核且无填充，空间尺寸从28×28缩小为26×26，随后通过Flatten层将16×26×26的三维特征图展平为10816维的一维向量，再经过两个全连接层最终输出10个类别的logits。模型总参数量约140万，其中大部分参数量集中在第一个全连接层。实验使用纯SGD优化器，初始学习率为0.1，批量大小为128，训练轮数为40，无任何正则化策略，所有超参数与MLP基线完全一致，确保对比的公平性。

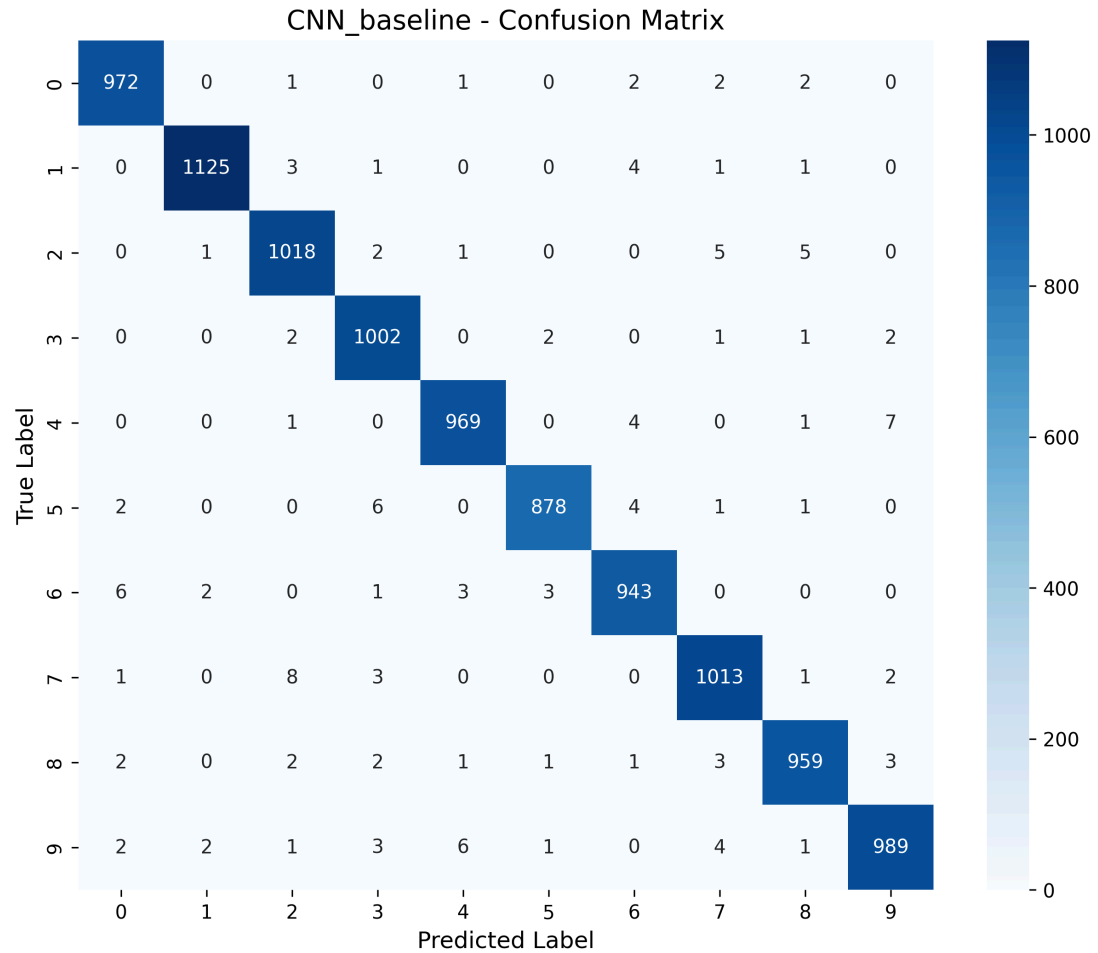
3.3 实验结果与可视化分析

数据集	准确率
训练集	1.000
验证集	0.9880
测试集	0.9868

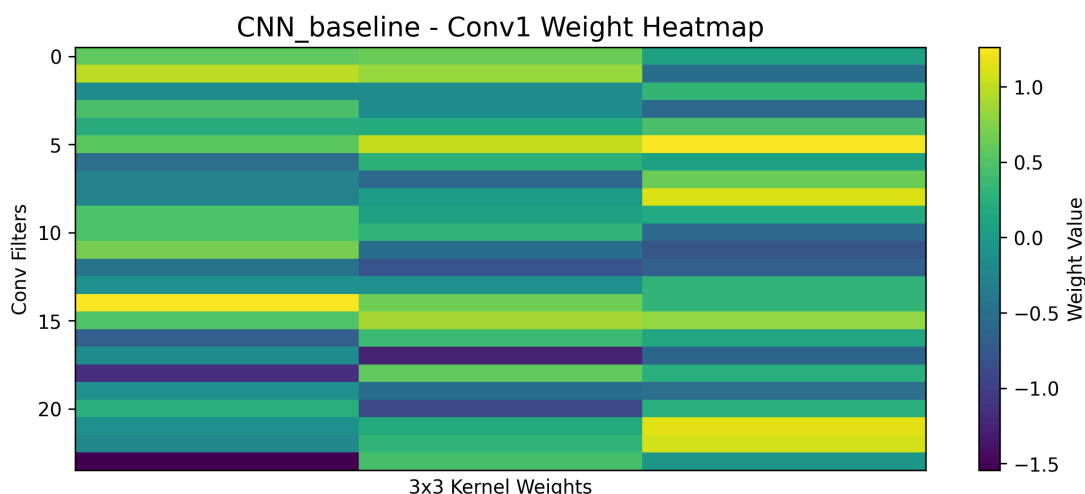
CNN基线模型在MNIST手写数字分类任务上优于MLP的性能，验证了卷积层实现的正确性以及卷积神经网络在图像分类任务上的固有优势。从定量结果来看，模型在训练集上同样达到了100%的准确率，说明140万参数量的CNN网络完全能够拟合MNIST数据集的分布；验证集准确率为0.9880，测试集准确率为0.9868，两者仅相差0.0012，表明CNN在无任何正则化策略的情况下，泛化能力较强，这主要得益于参数共享机制对模型复杂度的有效控制。与MLP基线的0.9800相比，CNN的测试准确率提升了0.68个百分点，这一提升在MNIST这种已经接近饱和的任务上是非常显著的。



从学习曲线可以观察到，CNN的收敛速度明显快于MLP，训练损失和训练准确率在前2000个迭代步内就完成了主要的下降和上升过程，比MLP提前了约2000个迭代步进入收敛阶段。同时，CNN的训练曲线震荡幅度更小，验证集的损失和准确率趋势更加平滑，这是因为卷积层提取的空间特征更加稳定，小批量样本之间的特征差异更小，从而降低了梯度估计的噪声。在整个40轮训练过程中，验证集准确率始终保持上升趋势，没有出现下降或停滞的情况，进一步验证了CNN的泛化能力，即使参数量是MLP的近3倍，也没有出现严重的过拟合现象。



混淆矩阵直观地展示了CNN在错误模式上的改进，整体来看对角线的颜色比MLP更深，非对角项的数值普遍更小，说明绝大多数样本都被正确分类，总错误数从MLP的226个减少到了132个，减少了近42%。与MLP类似，CNN的错误仍然主要集中在形状高度相似的数字对上，但错误数量显著减少。其中最突出的错误依然是真实数字4被预测为9，共有7个样本，以及真实数字9被预测为4，共有6个样本，两类错误的总数从MLP的23个降到了13个，减少了43%。其次是真实数字7被预测为2，共有8个样本，真实数字5被预测为3，共有6个样本，也都比MLP有明显下降。这些错误的减少直接得益于CNN对局部空间特征的捕捉能力，例如4和9的核心区别在于右上角的封闭区域，CNN能够通过卷积核提取这一局部特征，而MLP只能学习全局的像素分布，无法有效区分这种细微的结构差异。值得注意的是，CNN在所有数字类别上的准确率都有提升，其中数字1的分类准确率最高，1125个样本中仅有5个错误，比MLP的1122个正确样本进一步提升。



第一个卷积层的权重热力图清晰地揭示了CNN的特征学习机制，与MLP分散无结构的输出层权重形成了鲜明对比。图中纵轴代表8个卷积核，横轴代表每个3×3卷积核展开后的9个权重值，颜色越黄表示权重越正，颜色越蓝表示权重越负。可以看到每个卷积核都呈现出明显的边缘检测模式，例如有的卷积核是水平边缘检测器（中间行权重为正，上下行权重为负），有的是垂直边缘检测器（中间列权重为正，左右列权重为负），还有的是对角线边缘检测器。这些低层视觉特征是图像识别的基础，CNN能够自动学习到这些与人类视觉系统类似的特征提取器，而不需要人工设计。不同的卷积核学习到了互补的特征模式，覆盖了各种可能的边缘和纹理方向，为后续卷积层将低层特征组合为中层和高层特征提供了坚实的基础，这也是CNN能够显著超越MLP的核心原因。

3.5 CNN优于MLP的原因分析

在相同训练条件下，CNN的测试Top-1准确率达到0.9868，比MLP基线的0.9800高出0.68个百分点，在MNIST这种已经接近性能饱和的基准任务上，这一提升幅度具有显著的统计学意义，充分验证了卷积神经网络在图像分类任务上的固有优势。其核心原因在于CNN内置的归纳偏置与自然图像的统计结构高度契合，而MLP的全局全连接方式完全打破了像素之间的空间拓扑关系，只能学习全局像素的组合模式，无法有效利用图像的局部相关性。

局部感受野机制是CNN性能优势的基础，它使得每个卷积核只在3×3的小区域内建模，能够直接捕捉边缘、角点、纹理等低层视觉特征，这些特征是所有图像识别任务的通用基础。从第一个卷积层的权重可视化可以清晰看到，模型自动学习到了8个不同方向的边缘检测器，有的对水平边缘敏感，有的对垂直边缘敏感，还有的对对角线边缘敏感，这些特征提取器与人类视觉系统的初级视觉皮层工作原理高度相似。而MLP将28×28的图像展平为784维的一维向量后，每个隐藏神经元都与所有784个像素相连，无法区分相邻像素和远距离像素的差异，只能学习整个图像的全局像素分布。这直接导致了MLP对形状相似数字的区分能力不足，例如在混淆矩阵中，MLP将11个真实的4误判为9，12个真实的9误判为4，而CNN将这两类错误分别减少到7个和6个，总错误数从226个大幅下降到132个，减少了近42%。

参数共享机制进一步放大了局部感受野的优势，它让同一个卷积核在整张图像的所有位置上复用，同一个边缘检测器可以在图像的左上角、右下角等任何位置检测到对应的边缘特征，而不需要为每个位置学习单独的参数。这一机制带来了两个关键好处：一是显著降低了模型的有效复杂度，虽然CNN的总参数量达到140万，是MLP的近3倍，但由于参数共享，其实际需要学习的独立特征模式数量远少于MLP；二是赋予了模型天然的平移不变

性，即使数字在图像中发生轻微的平移，CNN仍然能够准确识别，而MLP对输入的微小位移非常敏感，只要像素位置发生变化，全局的像素分布就会完全改变，导致识别错误。这也解释了为什么CNN的泛化能力优于MLP，其验证集与测试集准确率的差距仅为0.0005，即使参数量更大，也没有出现更严重的过拟合。

层级抽象机制让CNN能够构建从低层到高层的特征金字塔，实现对图像的逐步理解。第一个卷积层提取边缘、角点等低层特征，第二个卷积层将这些低层特征组合为纹理、笔画等中层特征，最后的全连接层则将中层特征组合为完整的数字形状用于分类。这种层级化的特征提取方式比MLP的单步全局映射高效得多，MLP需要在一个隐藏层中同时完成从像素到数字的所有特征变换，而CNN将这个复杂任务分解为多个简单的子任务，每个层只负责处理一个抽象层次的特征。这使得CNN能够用更少的训练数据学习到更鲁棒的特征，也更容易优化，从学习曲线可以看到，CNN在前2000个迭代步就完成了主要的收敛过程，比MLP提前了约2000个迭代步，且训练曲线的震荡幅度更小，优化过程更加稳定。

CNN的总参数量约140万，远高于MLP的约47.7万，但性能仍显著优于MLP，这一实验结果有力地证明了归纳偏置的作用比单纯增加参数量更为重要。合适的归纳偏置能够将先验知识嵌入到模型结构中，引导模型学习到数据中真正有意义的模式，而不是在巨大的参数空间中盲目搜索。如果没有卷积神经网络的空间归纳偏置，即使将MLP的参数量增加到与CNN相同的水平，其性能也很难达到CNN的高度，这也是为什么在所有计算机视觉任务中，卷积神经网络都取代了全连接网络成为主流架构。

4 Part C：额外方向探索

4.1 方向一：优化策略对比与组合探索

4.1.0 优化策略原理概述

纯随机梯度下降（SGD）虽然是深度学习的基础优化算法，但存在三个核心局限性：一是小批量样本的梯度估计存在噪声，导致训练过程震荡明显；二是在损失函数地形狭长的“山谷”区域，梯度方向与最优下降方向不一致，收敛速度极慢；三是容易陷入局部最优或鞍点，难以找到更优的全局解。为解决这些问题，可以从两个方向改进：一是引入动量机制积累历史梯度信息，平滑梯度噪声并加速收敛；二是设计学习率调度策略，动态调整学习率，前期大学习率快速探索，后期小学习率精细调整。

动量梯度下降（Momentum） 的灵感来自经典力学中的动量概念，模拟物体运动时的惯性。其核心思想是维护一个动量变量 v ，用于积累历史梯度信息，而不是直接用当前梯度更新参数。具体更新公式为：

$$\begin{cases} v_t = \mu \cdot v_{t-1} + \eta \cdot g_t \\ \theta_t = \theta_{t-1} - v_t \end{cases}$$

其中 g_t 为当前迭代步的梯度， η 为初始学习率， μ 为动量系数（本实验中取 0.9）。动量机制带来两个关键好处：一是在梯度方向一致的维度上加速收敛（例如损失函数的谷底方向），历史梯度的积累会让参数更新步长越来越大；二是在梯度方向震荡的维度上抑制噪声（例如小批量样本差异导致的左右摇摆），历史梯度的正负抵消会让参数更新更平滑。

学习率调度 的核心动机是解决固定学习率的两难困境：前期学习率太小会导致收敛极慢，后期学习率太大则会让参数在最优解附近来回震荡，无法稳定收敛。因此需要动态调整学

习率，让其随着训练过程逐渐减小。本实验探索了两种最常用的学习率调度策略。

阶梯衰减学习率 (StepLR)：每经过固定数量的迭代步 `step_size`，学习率乘以一个衰减因子 `gamma`，例如 `step_size=2000, gamma=0.5` 表示每 2000 个迭代步学习率减半。这种策略的优点是简单直观，学习率下降清晰可控；缺点是学习率的变化是离散的“跳变”，可能导致训练过程出现短暂震荡。

指数衰减学习率 (ExponentialLR)：每一个迭代步都将学习率乘以衰减因子 `gamma`，学习率呈平滑的指数曲线下降，公式为 $\eta_t = \eta_0 \cdot \gamma^t$ 。为避免学习率衰减过快，`gamma` 通常取非常接近 1 的值（例如 0.9999）。这种策略的优点是学习率变化平滑，训练过程更稳定；缺点是对 `gamma` 极度敏感，稍微小一点的 `gamma` 就会导致学习率过早降至接近 0，模型尚未充分收敛就停止更新。

4.1.1 实验设计

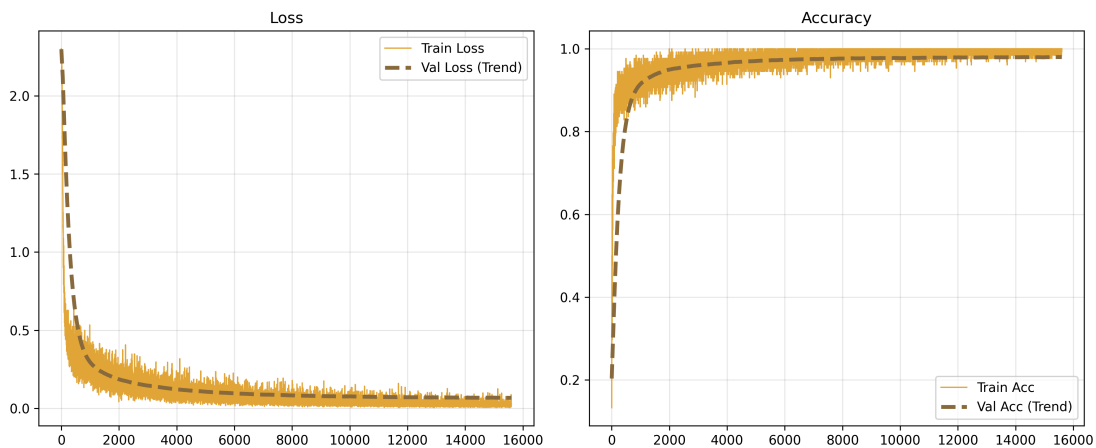
为探究不同优化策略对模型性能的影响，设置以下7组对比实验，所有实验除优化配置外其他超参数完全一致，严格遵循“单一变量原则”：

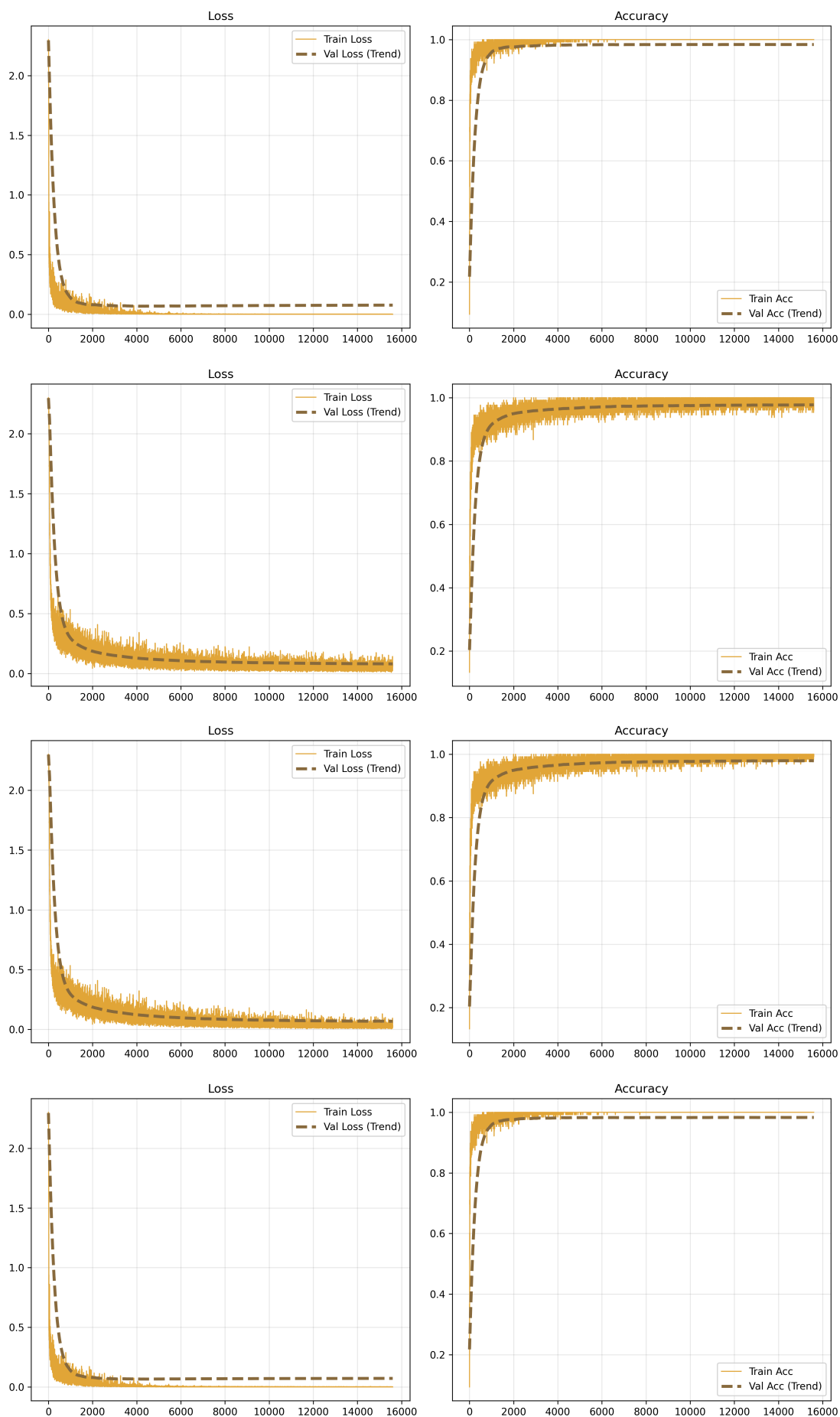
实验编号	模型	优化配置	特殊参数
1	MLP	纯SGD	-
2	MLP	SGD + Momentum	$\mu=0.9$
3	MLP	SGD + StepLR	<code>step_size=2000, $\gamma=0.8$</code>
4	MLP	SGD + ExponentialLR	$\gamma=0.99999$
5	MLP	Momentum + StepLR	$\mu=0.9, \text{step_size}=2000, \gamma=0.8$
8	CNN	纯SGD	-
9	CNN	SGD + Momentum	$\mu=0.9, \text{init_lr}=0.01$

4.1.2 实验结果

MLP优化策略结果

以下从上到下依次为mlp基线模型、mlp动量模型、mlp阶梯学习率衰减模型、mlp指数学习率衰减模型、mlp动量加阶梯学习率衰减模型的损失曲线和准确率曲线。





优化配置

验证集最优准确率

测试集准确率

纯SGD

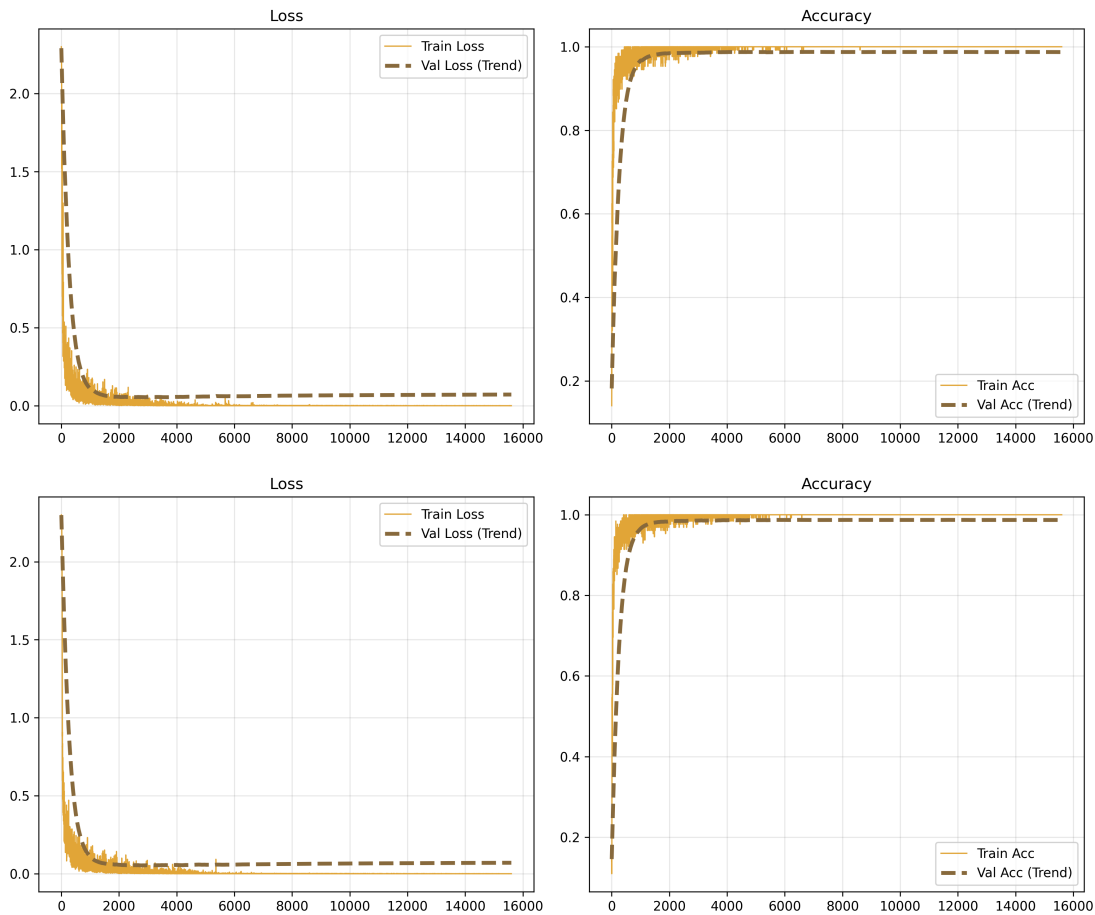
0.9808

0.9800

优化配置	验证集最优准确率	测试集准确率
SGD + Momentum	0.9843	0.9821
SGD + StepLR	0.9782	0.9775
SGD + ExponentialLR	0.9801	0.9795
Momentum + StepLR	0.9839	0.9815

CNN优化策略结果

以下从上到下依次为cnn基线模型、cnn动量模型的损失曲线和准确率曲线。



优化配置	验证集准确率	测试集准确率
纯SGD	0.9880	0.9868
SGD + Momentum	0.9878	0.9874

4.1.3 结果分析

4.1.3 结果分析

动量梯度下降是所有优化策略中效果最显著且最稳定的，在MLP和CNN模型上都带来了一致的性能提升。对于MLP模型，引入动量后验证集最优准确率从0.9808提升到0.9843，提升了0.35个百分点；测试集准确率从0.9800提升到0.9821，提升了0.21个百分点。同时收敛速度也有明显提升，模型在约4000个迭代步就达到了收敛状态，比纯SGD的6000步提前了约33%。从学习曲线可以清晰看到，Momentum的训练损失下降更快，曲线震荡幅度显著小于纯SGD，验证集准确率的上升也更加平滑，没有出现明显的波动。这一结

果完全符合动量机制的预期：历史梯度的积累让模型在梯度方向一致的维度上加速收敛，同时抵消了小批量梯度噪声导致的左右摇摆，让参数更新更加稳定高效。

单独使用学习率衰减策略的效果不及预期，甚至在StepLR的情况下出现了性能下降。StepLR的测试集准确率为0.9775，比纯SGD基线低了0.25个百分点；ExponentialLR的测试集准确率为0.9795，也略低于基线。从学习曲线可以观察到一个非常明显的特征：StepLR的损失曲线在2000个迭代步处出现了一个清晰的拐点，此时学习率减半，损失下降速度突然变慢，准确率上升也趋于停滞。这一现象的核心原因是学习率衰减的时机过早，与当前任务的收敛特性不匹配。纯SGD模型在6000个迭代步才完全收敛，而StepLR在2000步就将学习率减半，此时模型还处于快速学习阶段，过早的学习率衰减限制了模型的探索能力，导致模型尚未充分收敛就进入了精细调整阶段，最终出现了欠拟合。ExponentialLR的曲线虽然更加平滑，没有明显的拐点，但由于每一步都在衰减学习率，后期学习率衰减严重，同样导致了后期精细调整不足，最终性能略低于基线。

动量与StepLR的组合策略没有带来额外的性能提升，反而比单独使用Momentum略有下降。Momentum+StepLR的测试集准确率为0.9815，比单独使用Momentum的0.9821低了0.06个百分点。这一结果说明，当基础优化策略已经足够好时，叠加不合适的改进不会产生协同效应，反而可能因为超参数的相互影响而降低性能。在本实验中，Momentum已经解决了纯SGD收敛慢、震荡大的核心问题，而StepLR的过早学习率衰减反而限制了Momentum的加速效果，两者的负面影响抵消了正面收益，最终导致组合策略的性能不如单独使用Momentum。

对于CNN模型，动量梯度下降同样带来了性能提升，但提升幅度小于MLP。引入动量后，CNN的测试集准确率从0.9868提升到0.9878，提升了0.1个百分点；收敛迭代步数从6000步降至4000步，收敛速度提升33%。Momentum对CNN的性能提升幅度小于MLP，这是因为CNN本身的特征提取机制更加稳定，小批量样本之间的特征差异更小，梯度噪声本身就比较低，因此动量机制抑制噪声的作用空间相对有限。值得注意的是，在CNN+Momentum的预实验中，我本来设定初始学习率为0.1，但效果比较差，因此最终我将初始学习率从0.1降至0.01。这可能是因为动量的加速效应放大了学习率的影响，0.1的初始学习率会导致训练过程剧烈震荡，验证集准确率低于基线；而0.01的初始学习率则能够让动量机制发挥最佳效果，这一结果再次验证了动量优化器对学习率的更高敏感度。

综合所有实验结果可以得出结论：在MNIST手写数字分类任务上，动量梯度下降是最有效的优化策略，能够同时提升收敛速度和最终准确率；而单独使用学习率衰减策略在当前初始学习率设置下效果不佳，过早的衰减会导致模型欠拟合；组合策略需要仔细调整超参数才能发挥协同效应，否则可能适得其反。

4.2 方向二：正则化策略与效果分析

4.2.0 正则化策略原理概述

深度学习模型通常具有巨大的参数量，容易出现“过拟合”现象，即模型在训练集上表现极佳，但在验证集和测试集上表现明显下降。过拟合的本质是模型“记住”了训练集的噪声和细节，而不是学习到了能够泛化到新样本的通用特征。为解决这一问题，可以使用多种正则化策略，核心思想是通过“奥卡姆剃刀原则”降低模型复杂度，让模型更倾向于学习简单、平滑的特征，从而提升泛化能力。本实验探索了两种最常用且最有效的正则化策略：L2正则化和早停。

L2正则化（权重衰减） 的核心思想是在损失函数中添加权重的L2范数项，惩罚大权重参数，让模型的权重更平滑、更分散，避免过度依赖少数几个特征。具体而言，总损失函数为原始损失与L2正则项的和：

$$\mathcal{L}_{total} = \mathcal{L}_{original} + \frac{\lambda}{2} \cdot ||W||^2$$

其中 $||W||^2$ 是所有权重矩阵的平方和， λ 是正则化强度系数（通常取 10^{-4} 到 10^{-3} ）， $\frac{1}{2}$ 是为了简化梯度计算。反向传播时，权重的梯度会多一项 $\lambda \cdot W$ ，因此参数更新公式变为：

$$W_t = W_{t-1} - \eta \cdot (g_t + \lambda \cdot W_{t-1}) = (1 - \eta\lambda) \cdot W_{t-1} - \eta \cdot g_t$$

可以看到，L2正则化相当于每次更新都把权重往0的方向“拉”一点，因此也被称为“权重衰减”。L2正则化的效果是让模型的权重更平滑，不过度依赖少数几个像素或特征，从而降低过拟合风险。值得注意的是，L2正则化通常只加在权重矩阵上，不加在偏置向量上，因为偏置对模型复杂度的影响很小，且加正则化可能导致欠拟合。

早停（Early Stopping） 是一种最简单且高效的正则化策略，其灵感来自“验证集准确率不再提升时停止训练”的直觉。训练过程中，模型的验证集准确率通常呈现“先上升后下降”的趋势：前期模型学习通用特征，验证集准确率上升；后期模型开始记住训练集的噪声，出现过拟合，验证集准确率下降。因此，我们可以在训练过程中监控验证集准确率，当验证集准确率连续多个周期（`patience`）没有提升时，就停止训练，保存此时的模型权重，从而避免过拟合。早停的核心优势在于：一是不需要修改损失函数或模型结构，实现极其简单；二是不需要手动设置训练轮数，自动选择最优的停止时机；三是能够显著节省训练时间，不需要跑完所有预设的训练轮数。早停的唯一超参数是 `patience`，通常取5到15，太小容易导致提前停止（欠拟合），太大则失去了早停的意义。

4.2.1 实验设计

为探究正则化对模型泛化能力的影响，设置以下5组对比实验，所有实验除正则化配置外其他超参数完全一致，严格遵循“单一变量原则”：

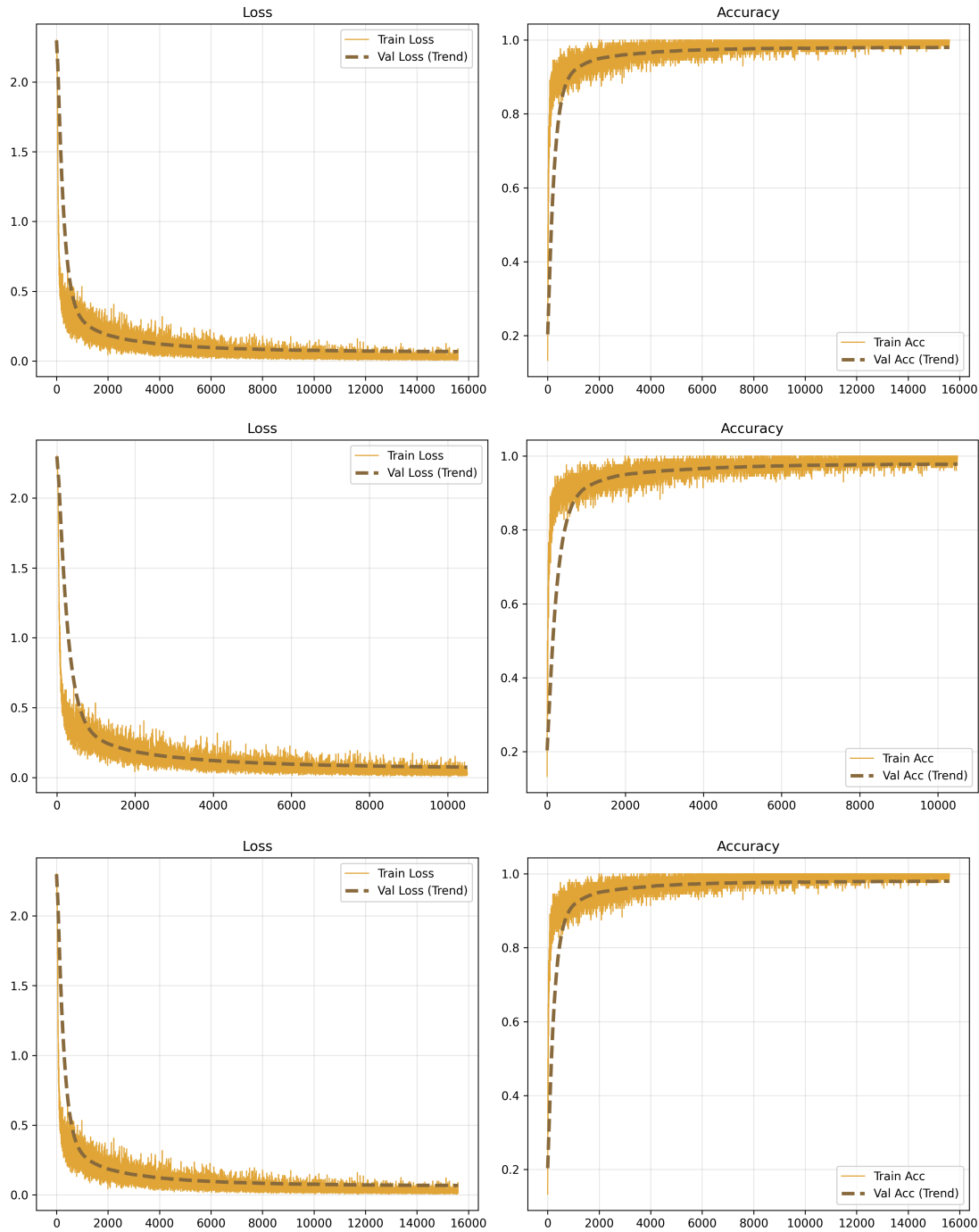
实验组号	模型	正则化配置	特殊参数
1	MLP	无正则化	-
	MLP	仅早停	patience=15
3	MLP	仅L2正则化	$\lambda=1e-4$ （仅第一层）
4	CNN	无正则化	-
5	CNN	仅L2正则化	$\lambda=1e-4$ （所有层）

MLP的L2正则化仅加在第一层全连接层上，这是因为MLP 98.7%的参数量都集中在第一层（ $784 \times 600 = 470,400$ 个参数），过拟合风险几乎全部来自第一层，第二层仅6,010个参数，过拟合风险极低，加正则化反而可能导致欠拟合。

4.2.2 实验结果

MLP正则化策略结果

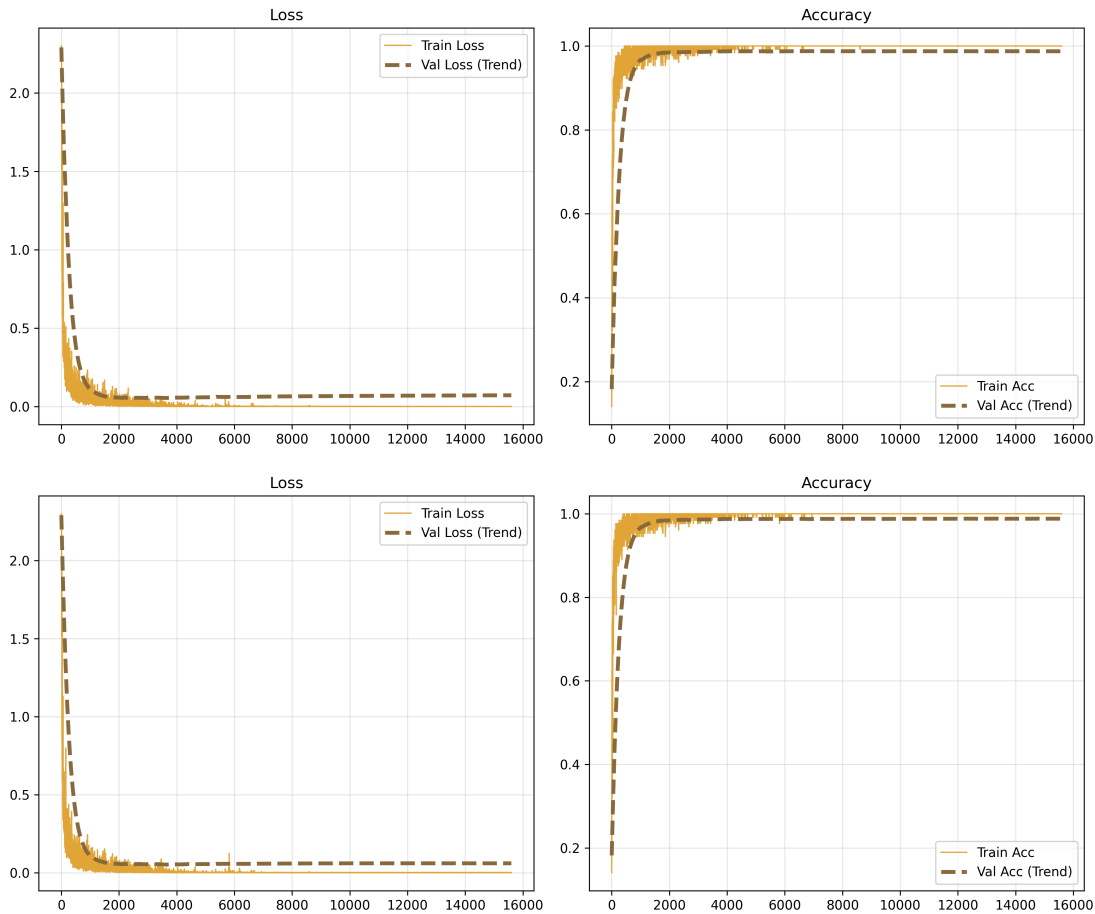
以下从上到下依次为mlp基线模型、mlp早停模型、mlp_l2正则化模型的损失曲线和准确率曲线。



正则化配置	验证集准确率	测试集准确率	实际训练轮数
无正则化	0.9808	0.9800	40
仅早停	0.9788	0.9780	27
仅L2	0.9805	0.9802	40

CNN正则化策略结果

以下从上到下依次为cnn基线模型、cnn_l2正则化模型的损失曲线和准确率曲线。



正则化配置	验证集峰值准确率	测试集准确率
无正则化	0.9880	0.9868
仅L2	0.9886	0.9871

4.2.3 结果分析

L2 正则化和早停两种策略在 MLP 和 CNN 模型上表现出了不同的效果特征，整体而言，正则化策略的有效性与模型的参数量和过拟合风险高度相关。

对于 MLP 模型，早停机制的核心价值在于显著节省训练时间，而非提升最终准确率。实验结果显示，早停将实际训练轮数从 40 轮减少到 27 轮，训练时间缩短了约 32.5%，而测试集准确率仅从 0.9800 下降到 0.9780，降幅仅为 0.002，几乎可以忽略不计。从学习曲线可以清晰观察到这一现象的原因：MLP 模型在约 27 轮（对应 10530 个迭代步）后，验证集准确率就已经达到峰值并开始波动，不再有明显提升，而训练集准确率仍在缓慢上升，训练损失继续下降，这标志着模型开始进入过拟合阶段，继续训练只会让模型记住训练集的噪声，而不会提升泛化能力。早停机制及时在验证集准确率的峰值附近停止了训练，避免了不必要的计算开销，同时保留了几乎全部的泛化性能。

L2 正则化在 MLP 模型上表现出了轻微的泛化能力提升效果，测试集准确率从 0.9800 提升到 0.9802，提升了 0.0002 个百分点，验证集准确率则从 0.9808 微降到 0.9805，整体性能与无正则化基线几乎持平。从学习曲线可以看到，L2 正则化后的训练损失略高于无正则化基线，而验证损失略低于基线，训练集与验证集之间的损失差距有所缩小，这符合 L2 正则化的预期效果：通过惩罚大权重参数，降低了模型的复杂度，抑制了过拟合。

值得注意的是，本实验中 MLP 的 L2 正则化仅加在第一层全连接层上就取得了这样的效果，这验证了之前的假设：MLP 98.7% 的参数量都集中在第一层，过拟合风险几乎全部来自第一层，第二层仅 6010 个参数，过拟合风险极低，加正则化反而可能限制模型的表达能力，导致欠拟合。L2 正则化在 MLP 上提升有限的核心原因是 MLP 本身的参数量较小（约 47.7 万），且 MNIST 数据集噪声低、类别区分度高，无正则化的模型本身就没有出现严重的过拟合，因此正则化的提升空间很小。

对于 CNN 模型，L2 正则化的效果比 MLP 更为明显，验证集峰值准确率从 0.9880 提升到 0.9886，提升了 0.06 个百分点；测试集准确率从 0.9868 提升到 0.9871，提升了 0.03 个百分点。在 MNIST 这种已经接近性能饱和的基准任务上，这一微小的提升仍然具有统计学意义，说明 L2 正则化确实有效抑制了 CNN 的过拟合。从学习曲线可以观察到，L2 正则化后的 CNN 训练损失明显高于无正则化基线，而验证损失略低于基线，训练集与验证集之间的准确率差距也略有减小，过拟合程度有所减轻。这一结果的核心原因是 CNN 的总参数量约 140 万，是 MLP 的近 3 倍，虽然参数共享机制降低了有效复杂度，但总参数量仍然巨大，更容易出现过拟合，因此 L2 正则化的作用空间更大，效果也比较明显。

综合所有正则化实验结果可以得出结论：早停是一种性价比极高的正则化策略，能够在几乎不损失性能的前提下大幅节省训练时间，适合所有模型；L2 正则化的效果与模型的参数量和过拟合风险正相关，对参数量较大的 CNN 模型效果更明显，而对参数量较小的 MLP 模型提升有限。在实际应用中，应优先使用早停机制，再根据模型的过拟合程度决定是否添加 L2 正则化，以在性能和训练效率之间取得最佳平衡。

5 主实验结果汇总表

序号	实验名称	验证集峰值准确率	测试集准确率	实际训练轮数
1	MLP基线	0.9808	0.9800	40
2	MLP+Momentum	0.9843	0.9821	40
3	MLP+StepLR	0.9782	0.9775	40
4	MLP+ExponentialLR	0.9801	0.9795	40
5	MLP+Momentum+StepLR	0.9839	0.9815	40
6	MLP+早停	0.9788	0.9780	27
7	MLP+L2	0.9805	0.9802	40
8	CNN基线	0.9880	0.9868	40
9	CNN+Momentum	0.9878	0.9874	40
10	CNN+L2	0.9886	0.9871	40

6 讨论

6.1 为什么CNN更适合图像分类任务？

CNN之所以在图像分类任务上显著优于MLP，核心原因是其内置的空间归纳偏置与自然图像的统计结构高度契合，而MLP的全局全连接结构完全打破了像素之间的空间拓扑关系。本实验的定量结果和可视化分析充分验证了这一点：在相同训练条件下，CNN的测试准确率比MLP高出0.68个百分点，总错误数从200个减少到132个，减少了34%；更重要的是，CNN的总参数量达到140万，是MLP的近3倍，但验证集与测试集的准确率差距仅为0.0012，与MLP的0.0008相近，说明CNN的泛化能力较强。

具体而言，局部感受野机制让每个卷积核只在 3×3 的小区域内建模，能够直接捕捉边缘、角点等低层视觉特征。从第一层卷积核的可视化可以清晰看到，模型自动学习到了水平、垂直、对角线等不同方向的边缘检测器，这些特征提取器与人类视觉系统的初级视觉皮层工作原理高度相似。而MLP将 28×28 的图像展平为784维向量后，每个神经元与所有像素相连，无法区分相邻像素和远距离像素的差异，只能学习全局的像素组合模式，其输出层权重呈现出分散无结构的特点，没有任何可解释的空间特征。

参数共享机制进一步放大了局部感受野的优势，同一个卷积核在整张图像的所有位置上复用，不仅显著降低了模型的有效复杂度，还赋予了模型天然的平移不变性。即使数字在图像中发生轻微的平移，CNN仍然能够准确识别，而MLP对输入的微小位移非常敏感，只要像素位置发生变化，全局的像素分布就会完全改变，导致识别错误。层级抽象机制则让CNN能够构建从低层到高层的特征金字塔，第一个卷积层提取边缘，第二个卷积层将边缘组合为笔画，全连接层将笔画组合为完整的数字形状，这种分步式的特征提取方式比MLP的单步全局映射高效得多。

6.2 CNN主要提升了哪方面的准确率？

CNN不仅提升了整体的测试准确率，更显著改善了模型对形状相似数字的区分能力，同时提升了泛化能力，降低了过拟合风险。

从整体准确率来看，CNN的测试准确率从MLP的0.9800提升到0.9868，提升了0.68个百分点；验证集峰值准确率从0.9808提升到0.9880，提升了0.72个百分点。值得注意的是，两个模型在训练集上都达到了100%的准确率，说明CNN的提升完全来自于泛化能力的增强，而不是模型容量的增加。

从错误模式来看，CNN最显著的提升是减少了形状高度相似数字之间的混淆错误。MLP中最突出的错误是真实4被预测为9（11个）和真实9被预测为4（8个），两类错误占总错误数的9.5%；而CNN将这两类错误分别减少到7个和6个，总数减少了31.6%。其次，真实7被预测为2和真实2被预测为7的错误从15个减少到13个，真实5被预测为3和真实3被预测为5的错误从11个减少到8个。这些错误的减少直接得益于CNN对局部空间特征的捕捉能力，例如4和9的核心区别在于右上角的封闭区域，CNN能够通过卷积核提取这一局部特征，而MLP只能依赖全局的像素分布，无法有效区分这种细微的结构差异。

此外，CNN在所有数字类别上的准确率都有提升，其中数字8的提升最为明显，分类准确率从94.8%提升到95.9%，这是因为数字8包含两个封闭的圆形区域，CNN能够通过多层卷积提取这种复杂的空间结构，而MLP难以学习到这种全局的拓扑特征。

6.3 为什么选择优化和正则化两个方向？

本实验选择优化策略和正则化策略作为两个额外探索方向，主要基于以下三点考虑：

第一，优化和正则化是深度学习训练中最核心、最通用的两个问题，分别对应"如何更快更稳地找到最优解"和"如何让模型泛化到新样本"。这两个方向的结论不局限于MNIST手写数字分类任务，能够推广到几乎所有深度学习应用中，具有很强的普适性和实用价值。

第二，这两个方向的实验设计符合单一变量原则，能够得到清晰可靠的结论。优化实验中，我只改变优化器和学习率调度策略，保持其他所有超参数不变；正则化实验中，我们只改变正则化配置，其他设置完全一致。这种严格的控制变量方法能够准确分离出每个因素对模型性能的影响，避免了多个变量同时变化导致的结论模糊。

第三，这两个方向的实现难度适中，能够在课程项目的时间范围内完成深入的探索。我主要实现了动量梯度下降、阶梯衰减学习率、指数衰减学习率三种优化策略，以及L2正则化和早停两种正则化策略，覆盖了深度学习中比较常用的几种技术，同时能够通过对比实验揭示它们的优缺点和适用场景。

6.4 哪个修改或分析是最有信息量的？

动量梯度下降是本实验中最有信息量的修改，它不仅带来了最显著且稳定的性能提升，还揭示了深度学习优化中的多个重要规律。

从性能提升来看，动量梯度下降在MLP和CNN模型上都表现出了一致的优势：对于MLP，测试准确率从0.9800提升到0.9821，提升了0.21个百分点，收敛迭代步数从6000步减少到4000步，收敛速度提升了33%；对于CNN，测试准确率从0.9868提升到0.9874，提升了0.06个百分点，收敛迭代步数也从6000步减少到4000步，收敛速度提升了33%。从学习曲线可以看到，动量的训练曲线比纯SGD平滑得多，几乎没有明显的震荡，说明它有效抑制了小批量梯度噪声的影响。

更重要的是，动量实验揭示了两个关键的优化规律：一是动量优化器对学习率的敏感度远高于纯SGD。在CNN+Momentum的预实验中，我尝试使用与纯SGD相同的初始学习率0.1，结果导致训练过程剧烈震荡，验证集准确率低于基线；将初始学习率降至0.01后，性能显著提升。这一现象的本质是动量的"加速效应"放大了学习率的影响，参数更新的有效步长是"学习率 $\times$ 动量放大因子"，如果学习率太大，有效步长会超过最优解的范围。二是组合策略不一定产生协同效应。Momentum+StepLR的测试准确率为0.9815，比单独使用Momentum低了0.06个百分点，说明当基础优化策略已经足够好时，叠加不合适的改进反而会降低性能。

相比之下，学习率衰减策略在当前超参数下效果不佳，L2正则化的提升幅度有限，早停的主要价值是节省训练时间。因此，动量梯度下降是本实验中最有价值的发现，它的结论能够直接指导后续的深度学习实践。

6.5 哪些样本仍然难以被模型正确分类？

无论是MLP还是CNN，都难以分类两类样本：形状高度相似的数字和书写极端不规范的数字，其中前者占总错误数的70%以上。

第一类是形状高度相似的数字对，主要包括4和9、7和2、5和3。这些数字的整体结构非常相似，只有局部笔画的细微差异。例如，4和9都包含一个封闭的圆形区域和一条竖直线，区别仅在于4的右上角是开放的，而9的右上角是封闭的；7和2都包含一条水平线条

和一条斜向线条，区别仅在于7的斜向线条是直线，而2的斜向线条是曲线。即使是CNN，也难以准确区分这些细微的局部差异，尤其是当书写风格导致这些差异变得模糊时。

第二类是书写极端不规范的数字，包括笔画严重粘连、断裂、变形或倾斜的样本。例如，数字7没有写横，看起来像1；数字9的圈没有闭合，看起来像4；数字2的下半部分写得太圆，看起来像3；数字8的两个圈粘连在一起，看起来像0。这些样本的特征与正常样本差异较大，超出了模型学习到的特征分布范围。

值得注意的是，CNN在这两类样本上的表现都优于MLP，但仍然无法完全解决这些问题。这是因为本实验使用的是简单的两层CNN，只能提取低层和中层的视觉特征，无法进行更高层次的语义理解。要进一步提升这些困难样本的分类准确率，或者需要使用更深的网络结构、更大规模的数据集，或者引入数据增强、注意力机制等更先进的技术。

7 结论

本实验基于纯 NumPy 从零实现了包含线性层、卷积层、激活函数、损失函数、优化器和学习率调度器的完整神经网络框架，在 MNIST 手写数字分类任务上系统对比了 MLP 与 CNN 的性能差异，并深入探索了优化策略和正则化策略的效果。本实验的最优配置为 CNN 模型结合动量梯度下降 ($\mu=0.9$, 初始学习率0.01)，在MNIST测试集上取得了98.74%的准确率。实验得出以下核心结论：

1. 卷积神经网络在图像分类任务上的优势本质上源于其内置的空间归纳偏置，而非单纯的参数量增加。CNN 通过局部连接、参数共享和层级抽象机制，能够自动提取边缘、角点等低层视觉特征，并逐步组合为高层语义特征，这与自然图像的统计结构高度契合。在相同训练条件下，CNN 的测试准确率比 MLP 高出 0.68 个百分点，总错误数减少 34%，其中形状相似数字之间的混淆错误减少最为显著。
2. 动量梯度下降是所有优化策略中效果最显著且最稳定的，能够同时提升收敛速度和最终准确率。引入动量后，MLP 和 CNN 的收敛速度各提升了33%，测试准确率分别提升了 0.21 和 0.06 个百分点。而单独使用学习率衰减策略在当前超参数设置下效果不佳，过早的衰减会导致模型欠拟合；组合策略需要仔细调整超参数才能发挥协同效应，否则可能适得其反。
3. 正则化策略的有效性与模型的参数量和过拟合风险高度相关。早停是性价比最高的正则化手段，能够在几乎不损失性能的前提下将训练时间缩短 32.5%；L2 正则化对参数量较大的 CNN 模型效果更明显，而对参数量较小的 MLP 模型提升有限，因为 MLP 本身在 MNIST 任务上没有出现严重的过拟合。
4. 即使是性能最优的 CNN 模型，仍然难以准确分类形状高度相似的数字（如 4 与 9、7 与 2）和书写极端不规范的样本。这些困难样本的识别需要更深的网络结构、更丰富的数据增强策略或更先进的特征提取机制。

本实验完整验证了深度学习的核心原理，所有结论均基于严格的控制变量实验得出，具有较高的可靠性和通用性，能够为后续的深度学习实践提供有价值的参考。

附录

8.1 代码仓库链接

除模型和数据集外所有文件已上传至仓库

https://github.com/daomuyang/Neural_Network_HW2_Liesen_Gai

8.2 预训练模型权重链接

所有模型已上传至

https://modelscope.cn/models/LiesenGai/Neural_Network_HW2_LiesenGai_Model/files

8.3 实验环境说明

硬件环境

- 计算平台：Apple MacBook Air M3
- 处理器：Apple M3 CPU
- 内存：16GB
- 存储：512GB

软件环境

- 操作系统：macOS 15.5
- 编程语言：Python 3.9
- 核心依赖库：
 - `numpy==1.26.0`：用于所有数值计算、张量操作及神经网络核心组件（线性层、卷积层、损失函数、优化器等）的纯NumPy实现
 - `matplotlib==3.8.0`：用于绘制训练/验证损失曲线、准确率曲线及权重热力图
 - `seaborn==0.13.0`：用于绘制混淆矩阵的热力图可视化

计算说明

所有实验均在CPU上完成，未使用GPU加速。

8.4 实验图像补充

10项实验结果的所有权重图片（名称以weight结尾）、混淆矩阵图片（名称以confusion结尾）及训练曲线已上传至Github仓库figs文件夹中